

AULA PRÁTICA

Funções e Classes em JavaScript

Exemplos reais com mini-sistema de agendamento de serviços

Duração	≈ 2 horas
Público-alvo	Alunos da Fase 2 — Sistemas de Informação (iniciantes em JavaScript)
Ambiente sugerido	Node.js, navegador ou editor online
Produto da aula	Um mini-sistema simples de agendamento

Ideia central da aula

Funções organizam ações; classes organizam entidades do sistema. Ao combinar as duas coisas, começamos a pensar como desenvolvedor de sistemas reais.

1. Objetivos da aula

- Entender funções como blocos reutilizáveis para executar ações específicas.
- Criar funções com parâmetros e retorno usando exemplos de sistemas reais.
- Conhecer a sintaxe de arrow functions como alternativa moderna.
- Compreender objetos como estruturas que representam entidades do mundo real.
- Criar classes com constructor, atributos e métodos.
- Instanciar objetos com new e usar métodos para alterar ou consultar estados.
- Reconhecer e evitar erros comuns ao trabalhar com classes.
- Construir um mini-sistema simples de agendamento de serviços em JavaScript.

2. Roteiro previsto

Tempo	Momento	Conteúdo	Prática
0–10 min	Abertura	Demonstração do produto final + problema real	Discussão guiada
10–50 min	Funções	Formatação, validação, desconto, mensagem e arrow functions	Exercício 1
50–60 min	Objetos	Representação de cliente e serviço	Leitura de propriedades
60–95 min	Classes	Cliente, Serviço e Agendamento	Codificação guiada

Tempo	Momento	Conteúdo	Prática
95–100 min	Armadilhas	Erros comuns ao usar classes	Discussão
100–115 min	Desafio	Mini-sistema completo	Exercício final
115–120 min	Fechamento	Síntese, bônus com <code>forEach</code> e tarefa	Orientações finais

3. Contexto prático da aula

A aula será conduzida a partir de um cenário comum em sistemas web: um sistema simples de agendamento de serviços.

Cenário-problema

Um cliente solicita um atendimento. O sistema precisa registrar o cliente, escolher o serviço, calcular valor, aplicar desconto, confirmar o horário e exibir um resumo do agendamento.

Esse contexto pode representar uma barbearia, salão de beleza, clínica, oficina, assistência técnica, consultório, escritório ou qualquer negócio local que organize atendimentos por horário.

4. Abertura — Demonstrando o produto final

Antes de quebrar o sistema em partes, mostramos juntos no console o resultado final que vamos construir ao longo da aula. A ideia é criar uma âncora visual: os alunos veem para onde estão indo antes de começar a programar.

Rode o código abaixo no console (ou apenas leia em voz alta o que ele faz). Não se preocupe se a sintaxe ainda não fizer sentido — esse é o destino, não o ponto de partida.

```
// Visão geral – vamos construir cada peça desta aula
const cliente = new Cliente("Ana Souza", "66988887777", "Sinop");
const servico = new Servico("Manicure", 35, 60);
const agendamento = new Agendamento(cliente, servico, "12/05/2026",
  "09:00");

agendamento.confirmar();
agendamento.exibirResumo();

// Saída esperada:
// Resumo do Agendamento
// -----
// Cliente: Ana Souza
// Serviço: Manicure
```

```
// Valor: R$ 35
// Data: 12/05/2026 às 09:00
// Status: Confirmado
```

5. Parte 1 — Funções em JavaScript

Uma função é um bloco de código criado para executar uma tarefa específica. Em um sistema real, funções ajudam a evitar repetição e deixam o código mais organizado.

Quando uma ação aparece várias vezes no sistema, provavelmente ela deveria virar uma função. Por exemplo: formatar nome, validar telefone, calcular valor final ou gerar uma mensagem de confirmação.

5.1 Função para formatar nome do cliente

```
function formatarNome(nome) {
    return nome.trim().toUpperCase();
}

const nomeCliente = formatarNome(" maria aparecida ");
console.log(nomeCliente); // MARIA APARECIDA
```

O método `trim()` remove espaços extras antes e depois do texto, enquanto `toUpperCase()` transforma o nome em letras maiúsculas.

5.2 Função para validar telefone

Sistemas reais precisam validar dados antes de cadastrar ou processar informações. Veja primeiro a forma mais comum em código de iniciante:

```
// Versão verbosa (funciona, mas dá para melhorar)
function validarTelefone(telefone) {
    if (telefone.length >= 10) {
        return true;
    } else {
        return false;
    }
}
```

Agora repare: a expressão `telefone.length >= 10` já é, por si só, verdadeira ou falsa. O `if/else` é redundante. Podemos refatorar:

```
// Versão refatorada — mais curta e mais clara
function validarTelefone(telefone) {
    return telefone.length >= 10;
}

console.log(validarTelefone("66999998888")); // true
console.log(validarTelefone("12345")); // false
```

Por que isso importa?

Sempre que você escrever `if (algo) { return true } else { return false }`, pare. Você pode retornar diretamente a expressão. Esse é o primeiro hábito de quem programa de forma limpa.

5.3 Função para calcular valor com desconto

```
function calcularValorComDesconto(valor, percentualDesconto) {  
  const desconto = valor * percentualDesconto / 100;  
  return valor - desconto;  
}  
  
const valorFinal = calcularValorComDesconto(100, 10);  
console.log("Valor final: R$ " + valorFinal); // R$ 90
```

5.4 Função para gerar mensagem de confirmação

Aqui aparece um recurso muito útil do JavaScript moderno: template literals. Em vez de juntar texto com `+`, usamos crases (```) e `${variavel}` para inserir valores dentro do texto.

```
function gerarMensagemConfirmacao(nome, servico, data, horario) {  
  return `Olá, ${nome}! Seu agendamento para ${servico} foi confirmado  
para o dia ${data} às ${horario}.`;  
}  
  
const mensagem = gerarMensagemConfirmacao(  
  "Maria",  
  "Corte de cabelo",  
  "10/05/2026",  
  "14:00"  
);  
  
console.log(mensagem);  
// Olá, Maria! Seu agendamento para Corte de cabelo foi confirmado para  
o dia 10/05/2026 às 14:00.
```

Concatenação x template literal

Concatenação: `"Olá, " + nome + "! Você agendou " + servico`

Template literal: ``Olá, ${nome}! Você agendou ${servico}``

São equivalentes, mas a versão com crases é mais legível e tem menos chance de erro de aspas. Use template literals sempre que possível.

5.5 Arrow functions — uma forma mais curta

À medida que vocês forem programando mais, vão encontrar funções escritas de outra maneira: as arrow functions ("funções de seta"). É só uma sintaxe alternativa, mais curta, que vamos usar bastante daqui em diante (especialmente quando trabalharmos com eventos da página e listas).

```
// Função tradicional
function formatarNome(nome) {
  return nome.trim().toUpperCase();
}

// Mesma função, agora como arrow function
const formatarNome = (nome) => nome.trim().toUpperCase();

// Quando o corpo tem várias linhas, usamos chaves e return:
const calcularValorComDesconto = (valor, percentual) => {
  const desconto = valor * percentual / 100;
  return valor - desconto;
};
```

Por enquanto, basta saber que as duas formas fazem a mesma coisa. As diferenças mais profundas (como o comportamento do this) ficam para uma aula futura.

6. Exercício 1 — Funções aplicadas ao agendamento

Tempo sugerido: 15 minutos.

Crie um arquivo chamado agendamento-funcoes.js e implemente as funções abaixo. Tente usar template literals onde fizer sentido:

1. `formatarNome(nome)`: remove espaços extras e devolve o nome em letras maiúsculas.
2. `calcularHorarioTermino(horarioInicio, duracaoMinutos)`: recebe um horário no formato "HH:MM" e a duração em minutos. Retorna o horário previsto de término. Dica: usar `split(":")` para separar horas e minutos.
3. `formatarDataBR(dataISO)`: recebe uma data no formato "2026-05-10" e retorna no formato "10/05/2026".
4. `calcularValorFinal(valorServico, percentualDesconto)`: retorna o valor final com desconto.
5. `gerarMensagemAgendamento(nome, servico, data, horario)`: retorna uma mensagem de confirmação usando template literal.

Orientação didática

Testem cada função com `console.log`. O foco é entender entrada, processamento e saída. Quem terminar antes pode tentar reescrever pelo menos uma das funções como arrow function.

7. Parte 2 — Objetos antes de classes

Antes de vermos as classes, observem que um objeto agrupa dados relacionados a uma mesma entidade do sistema.

```
const cliente = {  
  nome: "Maria Silva",  
  telefone: "66999998888",  
  cidade: "Sinop"  
};  
  
console.log(cliente.nome);  
console.log(cliente.telefone);  
console.log(cliente.cidade);
```

Cliente, produto, serviço, pedido, usuário e agendamento são exemplos de entidades que podem ser representadas como objetos.

8. Parte 3 — Classes em JavaScript

Uma classe é um modelo para criar objetos. Ela define quais dados (atributos) e comportamentos (métodos) os objetos terão.

O que é o this?

Antes de ler o próximo código, segure essa ideia: `this` é a forma da classe se referir ao objeto específico que está sendo usado naquele momento. Quando você escreve `this.nome`, está dizendo "o nome deste objeto aqui". Sem isso, todos os clientes compartilhariam o mesmo nome, o que não faria sentido.

8.1 Classe Cliente

```
class Cliente {  
  constructor(nome, telefone, cidade) {  
    this.nome = nome;  
    this.telefone = telefone;  
    this.cidade = cidade;  
  }  
  
  exibirDados() {  
    console.log(`Cliente: ${this.nome}`);  
    console.log(`Telefone: ${this.telefone}`);  
    console.log(`Cidade: ${this.cidade}`);  
  }  
}  
  
const cliente1 = new Cliente("Maria Silva", "66999998888", "Sinop");  
cliente1.exibirDados();
```

Cliente é o modelo, cliente1 é o objeto criado, constructor recebe os dados iniciais e `this` representa o próprio objeto.

8.2 Classe Servico

```
class Servico {
  constructor(nome, valor, duracaoMinutos) {
    this.nome = nome;
    this.valor = valor;
    this.duracaoMinutos = duracaoMinutos;
  }

  aplicarDesconto(percentual) {
    const desconto = this.valor * percentual / 100;
    return this.valor - desconto;
  }

  exibirServico() {
    console.log(`Serviço: ${this.nome} | R$ ${this.valor} | $
    {this.duracaoMinutos} min`);
  }
}

const servico1 = new Servico("Corte de cabelo", 50, 40);
servico1.exibirServico();
console.log(`Com desconto: R$ ${servico1.aplicarDesconto(10)}`);
```

Repare que aplicarDesconto faz a mesma coisa que a função calcularValorComDesconto da Parte 1. A diferença é que agora o método pertence ao próprio serviço — ele já sabe qual é o seu valor.

8.3 Classe Agendamento

```
class Agendamento {
  constructor(cliente, servico, data, horario) {
    this.cliente = cliente;
    this.servico = servico;
    this.data = data;
    this.horario = horario;
    this.status = "Agendado";
  }

  cancelar() {
    this.status = "Cancelado";
  }

  confirmar() {
    this.status = "Confirmado";
  }

  exibirResumo() {
    console.log("Resumo do Agendamento");
    console.log("-----");
    console.log(`Cliente: ${this.cliente.nome}`);
    console.log(`Telefone: ${this.cliente.telefone}`);
    console.log(`Serviço: ${this.servico.nome}`);
  }
}
```

```

    console.log(`Valor: R$ ${this.servico.valor}`);
    console.log(`Data: ${this.data} às ${this.horario}`);
    console.log(`Status: ${this.status}`);
  }
}

const cliente = new Cliente("Ana Souza", "66988887777", "Sinop");
const servico = new Servico("Manicure", 35, 60);
const agendamento = new Agendamento(cliente, servico, "12/05/2026",
  "09:00");

agendamento.exibirResumo();
agendamento.confirmar();
agendamento.exibirResumo();

```

Ponto importante: composição

A classe Agendamento recebe objetos criados por outras classes (Cliente e Servico). Esse padrão se chama composição e é uma das ideias mais usadas em sistemas reais. Em vez de uma classe gigante que faz tudo, temos classes pequenas que se combinam.

9. Armadilhas comuns ao usar classes

Antes do exercício final, vamos olhar para os erros que aparecem com mais frequência quando alguém está aprendendo classes. Saber reconhecê-los economiza horas de depuração.

9.1 Esquecer o new

```

// ❌ ERRADO
const cliente = Cliente("Ana", "66999998888", "Sinop");
// TypeError: Class constructor Cliente cannot be invoked without 'new'

// ✅ CORRETO
const cliente = new Cliente("Ana", "66999998888", "Sinop");

```

9.2 Esquecer o this dentro do método

```

// ❌ ERRADO – nome não existe nesse escopo
exibirDados() {
  console.log(`Cliente: ${nome}`);
}

// ✅ CORRETO – this aponta para o próprio objeto
exibirDados() {
  console.log(`Cliente: ${this.nome}`);
}

```


9.3 Confundir a classe com o objeto

```
// ❌ ERRADO – Cliente é o molde, não tem nome próprio
Cliente.exibirDados();

// ✅ CORRETO – chamamos o método em uma instância
cliente1.exibirDados();
```

10. Exercício 2 — Classe Cliente

Tempo sugerido: 10 minutos.

Crie uma classe Cliente com os seguintes atributos: nome, telefone e email. Depois, crie um método `exibirContato()` para mostrar os dados do cliente usando template literals.

Ao final, crie pelo menos dois clientes diferentes e chame o método `exibirContato()` para cada um.

11. Exercício 3 — Classe Produto para loja online

Tempo sugerido: 15 minutos.

Crie uma classe Produto com os atributos nome, preço e estoque. A classe deve possuir os seguintes métodos:

6. `vender(quantidade)`: diminui o estoque se houver quantidade suficiente. Caso contrário, exibe uma mensagem informando que o estoque é insuficiente.
7. `reporEstoque(quantidade)`: aumenta a quantidade em estoque.
8. `exibirProduto()`: mostra nome, preço e estoque atual.

Contextualização

Esse exercício pode ser ligado a uma loja online local, com produtos cadastrados e controle simples de estoque.

12. Exercício final — Mini-sistema de agendamento

Tempo sugerido: 15 minutos.

Crie um mini-sistema com três classes: Cliente, Serviço e Agendamento.

Classe	Atributos	Métodos
Cliente	nome, telefone	(opcional)
Serviço	nome, valor	<code>calcularValorComDesconto(percentual)</code>
Agendamento	cliente, serviço, data, horário, status	<code>confirmar()</code> , <code>cancelar()</code> , <code>exibirResumo()</code>

Depois, crie um cliente, um serviço, um agendamento, confirme o agendamento e exiba o resumo final no console.

13. Bônus — Trabalhando com vários agendamentos

Sistemas reais raramente lidam com um único agendamento. Eles trabalham com listas. Veja como exibir vários resumos de uma vez:

```
const agendamentos = [agendamento1, agendamento2, agendamento3];

agendamentos.forEach(a => a.exibirResumo());
```

O método `forEach` percorre cada item da lista e executa uma ação. Aquela função curta com seta passada para dentro do `forEach` é uma arrow function — e também é o que chamamos de callback. Vamos explorar isso a fundo nas próximas aulas, quando trabalharmos com eventos e manipulação da página.

14. Fechamento da aula

- Funções representam ações ou cálculos específicos.
- Template literals tornam a construção de mensagens muito mais legível.
- Arrow functions são uma sintaxe alternativa e enxuta para funções.
- Objetos representam dados relacionados a uma entidade.
- Classes funcionam como modelos para criar objetos com dados e comportamentos.
- A composição entre classes é o que dá origem a sistemas reais.
- Um sistema real começa com pequenas estruturas bem organizadas.

Frase para fechamento

Programar bem não é apenas fazer o código funcionar. É organizar o pensamento para que o código possa crescer, ser entendido e reutilizado.

15. Atividade para casa — Sistema de pedidos

Crie um sistema simples de pedidos para uma loja online com três classes: Cliente, Produto e Pedido.

- Cliente: nome, telefone e cidade.
- Produto: nome, preço e estoque; métodos `vender(quantidade)` e `exibirProduto()`.
- Pedido: cliente, produto, quantidade e status; métodos `calcularTotal()`, `confirmarPedido()` e `exibirResumo()`.

Use template literals nas mensagens. Quem se sentir confortável pode criar uma lista com vários pedidos e usar `forEach` para exibir todos os resumos.

Essa atividade prepara vocês para evoluir o exemplo em aulas futuras, usando formulário HTML, manipulação do DOM, `localStorage`, API com Express e banco de dados.

Próximas aulas

- Transformar o exemplo em uma página HTML com formulário de cadastro.
- Usar manipulação do DOM (e callbacks de eventos) para exibir o resumo do agendamento na tela.
- Salvar agendamentos no localStorage.